



DAIMON

D M N

Un protocollo deflazionario senza proprietario,
governato da una DAO, su BNB Chain

Nessun proprietario · Nessuna emissione

Floor di supply a 21 miliardi

Timelock pubblico di 7 giorni

Indice

1 Abstract	1
2 Il nome	1
2.1 Il concetto viene prima	1
2.2 Socrate — la guida che trattiene invece di comandare	2
2.3 Platone — la guida è scelta, prima che sorga la questione	2
2.4 Eraclito — la guida e il guidato sono la stessa cosa	2
2.5 Cosa ne consegue	2
3 Il problema	2
3.1 Un token con un proprietario è una promessa, non un sistema	2
3.2 La stessa struttura, su scala più grande	3
3.3 Cosa richiede l'alternativa	3
3.4 Una nota sul metodo	3
4 Da DMX a DMN	4
4.1 Cosa c'era prima	4
4.2 Cosa la sostituisce	6
4.3 Come funziona la migrazione	6
4.4 Cosa succede ai token mai riscattati	7
4.5 Perché una migrazione invece di rinunciare all'ownership	7
5 Architettura	7
5.1 Cinque contratti	7
5.2 Dove sta l'autorità	8
5.3 Le due eccezioni	8
6 Tokenomics	8
6.1 Supply	8
6.2 Il floor	8
6.3 Fee sulle transazioni	9
6.4 Reflection	9
6.5 Il ciclo di burn	9
6.6 Cosa succede al floor	10
7 Staking	10
7.1 Lock e peso	10
7.2 Reward in BNB	10
7.3 Checkpoint del potere di voto	11
7.4 Una scelta di design: il potere di voto non decade	11
7.5 Limiti	11
8 Governance	12
8.1 Il ciclo	12
8.2 Il potere di voto allo snapshot	12
8.3 Cosa conta per il quorum	12
8.4 Limiti alla governance stessa	13
8.5 Cosa può fare la governance	13
8.6 Il guardian	13
9 Caso di studio: Proposta #0	14
9.1 La decisione	14
9.2 La cronologia	14
9.3 La parte che conta di più	14
9.4 Verifica del risultato	15
9.5 La proposta non può essere eseguita due volte	15
9.6 Una proposta fallita	15
9.7 Cosa dimostra questo registro	16
10 Sicurezza	16
10.1 Modello di minaccia	16
10.2 Cosa è stato testato	16
10.3 Cosa abbiamo trovato noi stessi	17
10.4 Limiti noti	17
10.5 L'interfaccia non è il protocollo	17
10.6 Cosa è stato dimostrato on-chain	18
10.7 Audit esterno	18
11 Fondi e tesoreria	18
11.1 Due luoghi, due regole	18
11.2 Perché non tutto sotto governance	18
11.3 Cosa la governance controlla qui	19
11.4 Impegni	19
12 Roadmap	19
12.1 Fase 1 — Ora	19
12.2 Fase 2 — Daimon come protocollo DeFi	19
12.3 Fase 3 — Una tesoreria attiva	20
12.4 Fase 4 — Infrastruttura	20
12.5 La roadmap non è fissa	20
13 Cosa Daimon non è	21
14 Avvertenze legali	21

1. Abstract

Daimon è un token deflazionario governato interamente da chi lo possiede, deployato su BNB Chain. Non ha proprietario, non ha amministratore, e non ha funzione di emissione. La sua supply totale può solo diminuire — da un valore iniziale di 1.000 miliardi di token verso un floor immutabile di 21 miliardi, sotto il quale il codice rende ogni ulteriore burn matematicamente impossibile.

Ogni parametro che governa il protocollo — le fee sulle transazioni, il funzionamento dello staking, l'allocazione della tesoreria, e il codice stesso — può essere modificato soltanto attraverso un voto on-chain seguito da un timelock pubblico obbligatorio di sette giorni. Nessun individuo, inclusi coloro che lo hanno costruito, detiene il potere di alterare, mettere in pausa permanentemente, accelerare o aggirare quel processo. Non è un impegno che chiediamo di credere sulla parola: è una proprietà dei contratti deployati, verificabile da chiunque in qualsiasi momento.

Chi blocca i propri token riceve potere di voto proporzionale sia alla quantità sia alla durata del proprio impegno, insieme a reward pagati in BNB — finanziati dall'attività di scambio reale, mai da token di nuova emissione. Non esiste inflazione, perché non esiste alcun meccanismo in grado di produrla.

Daimon è la migrazione di un token esistente verso la completa decentralizzazione. Dove la versione precedente aveva un proprietario con controllo discrezionale e una fee dell'11% sulle transazioni, il nuovo protocollo non ha proprietario, ha una fee del 4% stabilita da un voto della community, e un tetto massimo scritto nel codice che nessun voto futuro potrà superare.

Il protocollo non promette rendimenti. Garantisce regole: una scarsità che non può essere diluita, decisioni che non possono essere prese in privato, e un sistema che continua a funzionare esattamente come specificato, che qualcuno lo stia sorvegliando o meno.

2. Il nome

Nella Grecia antica il *daimon* non era un demone. Era uno spirito guida — una forza divina inferiore che abitava lo spazio tra gli dèi e i mortali, né sopra l'umanità né sotto di essa, ma accanto.

La parola deriva da un verbo che significa *dividere, spartire, assegnare una parte*. Il daimon era colui che assegnava il destino: non un sovrano che lo imponeva, ma un custode che lo accompagnava.

Per i greci la felicità stessa prendeva il nome da questa relazione. *Eudaimonia* — letteralmente, *avere un buon daimon accanto*. La vita realizzata non era qualcosa concesso dall'alto; era il risultato di una buona guida, ben seguita.

I filosofi lo intesero ciascuno a modo proprio, e ogni lettura conta qui:

Socrate descriveva il suo *daimonion* come una voce interiore che non gli diceva mai cosa fare — solo quando fermarsi. Un freno, non un comando. Interveneva unicamente per impedire l'errore.

Platone, nel Mito di Er, scrisse che sono le anime a scegliere il proprio daimon prima di nascere. La guida non è assegnata dall'alto: è selezionata da chi verrà guidato.

Eraclito lo disse nel modo più netto: *il carattere è destino* — l'indole di una persona è il suo stesso daimon. Nulla di esterno. La guida e il guidato sono la stessa cosa.

2.1 Il concetto viene prima

Questo protocollo non è stato costruito e poi battezzato. Il nome non è un'etichetta applicata a posteriori a un sistema finito, e la corrispondenza descritta qui sotto non è una coincidenza notata in retrospettiva.

Il concetto di daimon è ciò che ha prodotto il progetto. Ogni decisione architettonica — cosa il protocollo può fare, cosa deve rifiutarsi di fare, dove è collocata l'autorità e dove è deliberatamente assente — è stata presa contro quell'idea. Il codice è il concetto scritto in Solidity.

Le tre letture sopra non sono decorazione. Ciascuna corrisponde a una proprietà specifica dei contratti deployati.

2.2 Socrate — la guida che trattiene invece di comandare

Il *daimonion* non diceva mai a Socrate cosa fare. Interveneva solo per fermarlo. Un segnale negativo: non direzione, ma freno.

```
if (block.timestamp < op.readyTimestamp) revert TooEarly();
```

Il timelock non contiene alcuna logica su cosa costituisca una buona decisione. Non approva, non consiglia, non valuta, non migliora. Non ha opinioni sulla proposta che gli passa attraverso. Il suo intero contributo è un rifiuto condizionato dal tempo.

La governance decide. Il timelock impedisce soltanto che qualcosa accada più in fretta di quanto possa essere esaminato. È un freno, e nient'altro — che è l'unica forma di autorità nel sistema a operare senza un voto.

2.3 Platone — la guida è scelta, prima che sorga la questione

Nel Mito di Er le anime scelgono il proprio daimon *prima* di nascere. La guida non è assegnata da un potere superiore; è scelta da chi verrà guidato, e scelta in anticipo.

```
votingPowerAt(voter, proposal.snapshotTimestamp)
```

Il potere di voto è valutato nel momento in cui una proposta è stata creata, non nel momento del voto. L'influenza su una decisione deriva da un impegno preso prima che quella decisione esistesse come questione.

L'effetto pratico è che il potere non può essere acquisito in reazione a qualcosa. Non si può vedere una proposta sgradita, comprare influenza, e usarla. La scelta precede la questione — che è precisamente ciò che Platone descriveva, e precisamente ciò che uno snapshot impone.

2.4 Eraclito — la guida e il guidato sono la stessa cosa

Il carattere è destino. Per Eraclito il daimon non era affatto esterno: l'indole di una persona è il suo stesso daimon. Non esiste un'entità separata.

```
GOVERNANCE_ROLE → DaimonTimelock  
(nessun altro detentore, in nessun contratto)
```

Non esiste alcuna autorità sopra il protocollo. Nessun proprietario, nessun amministratore, nessun consiglio, nessuna fondazione, nessun indirizzo che possa agire sul sistema dall'esterno. L'unico ruolo che esiste è detenuto da un contratto che non fa nulla se non eseguire ciò che la community ha già deciso.

Ciò che chi detiene sceglie è ciò che il protocollo diventa. Non perché promettiamo di rispettare le sue decisioni, ma perché non esiste alcun meccanismo attraverso cui qualcuno potrebbe non rispettarle. Il carattere della community è il destino del sistema — e l'assenza di qualsiasi altra autorità è ciò che rende quell'affermazione strutturale invece che aspirazionale.

2.5 Cosa ne consegue

Tre proprietà, tre letture, un concetto: un'autorità che spartisce invece di accumulare, trattiene invece di comandare, ed è scelta invece che imposta.

Tutto il resto di questo documento è l'elaborazione di quell'idea in meccanismi specifici. Dove i due sono mai entrati in conflitto durante lo sviluppo, ha vinto il concetto — anche quando ha reso il lavoro considerevolmente più difficile, come descrive la Sezione 4.5.

3. Il problema

3.1 Un token con un proprietario è una promessa, non un sistema

La maggior parte dei token viene deployata con un account amministrativo — il *proprietario* — che mantiene funzioni privilegiate. A seconda dell'implementazione, il proprietario può cambiare le fee, congelare i trasferimenti, escludere indirizzi dai limiti, prelevare fondi accumulati e in molti casi emettere nuovi token. Queste capacità non sono nascoste: sono scritte nel contratto, visibili a chiunque lo legga.

La questione non è se tali poteri esistano. È cosa si frappone tra quei poteri e il loro abuso.

In quasi ogni caso, la risposta è: le intenzioni del proprietario. Nient'altro. Nessuna procedura, nessun ritardo, nessun obbligo di annuncio, nessun modo per chi detiene di obiettare prima che accada. Il modello di sicurezza si riduce a una frase che compare, in qualche forma, in migliaia di progetti: *fidatevi di noi*.

Non è una critica all'onestà di un team in particolare. È un'osservazione sulla struttura. Un sistema la cui sicurezza dipende dalla buona fede continuativa di una singola chiave privata non è un sistema sicuro — è una scommessa su una persona. Le chiavi si perdono. Gli account vengono compromessi. Le persone cambiano idea, subiscono pressioni, o commettono errori alle tre di notte. E una scommessa che paga in modo affidabile per due anni resta una scommessa.

3.2 La stessa struttura, su scala più grande

Il modello non è esclusivo del crypto. È il modello del sistema finanziario in cui la maggior parte delle persone già vive.

Decisioni che determinano il valore dei risparmi delle persone comuni — quanta valuta emettere, quali commissioni applicare, quali condizioni modificare e quando — vengono prese da istituzioni che non consultano chi ne è colpito, non annunciano i cambiamenti in anticipo, e li spiegano dopo in un linguaggio costruito per scoraggiare l'esame. I depositanti non sono partecipanti; sono la materia trattata. L'asimmetria informativa non è un effetto collaterale del sistema: ne è una caratteristica.

L'inflazione è l'esempio più chiaro. Non è un fenomeno naturale. È una decisione, presa da un numero ristretto di persone, il cui effetto è ridurre il valore dei risparmi detenuti da tutti gli altri. Non richiede consenso, non offre possibilità di sottrarsi, e raramente viene descritta per ciò che è.

Il crypto avrebbe dovuto essere l'alternativa. Eppure un token il cui proprietario può cambiare le regole a piacimento ha semplicemente riprodotto la stessa struttura con un budget più piccolo: un'autorità opaca, decisioni discrezionali, e partecipanti che lo scoprono dopo.

3.3 Cosa richiede l'alternativa

Un'alternativa autentica non può basarsi su intenzioni migliori. Deve rendere le azioni problematiche strutturalmente impossibili.

Concretamente significa:

- **Nessuna funzione di emissione.** Non disattivata — assente. La diluizione non può essere votata, non può essere abilitata da un aggiornamento, non può essere introdotta da nessuno in nessuna circostanza, perché il codice per eseguirla non esiste.
- **Nessun account privilegiato.** Dopo il deploy, nessun wallet esterno detiene diritti amministrativi sul protocollo. Non chi ha fatto il deploy, non il team, non alcun individuo.
- **Nessun cambiamento silenzioso.** Ogni modifica passa da una proposta pubblica, un voto con una soglia minima di partecipazione, e un ritardo obbligatorio abbastanza lungo perché chiunque possa leggere cosa sta per accadere e agire di conseguenza.
- **Nessun parametro illimitato.** Persino la community, votando all'unanimità, non può alzare le fee sopra un tetto fissato nel codice, né bruciare la supply sotto il floor. Il sistema limita la propria stessa governance.
- **Nessuna fiducia richiesta.** Ogni affermazione in questo documento corrisponde a codice pubblico, deployato e verificabile. Nulla di ciò che è scritto qui chiede di essere creduto.

Daimon è un tentativo di costruire esattamente questo, e di farlo per una community esistente: non come un nuovo lancio, ma come la migrazione di un token che aveva già un proprietario verso un sistema che non ne ha.

3.4 Una nota sul metodo

Questo documento contiene una critica al potere finanziario concentrato. Non contiene alcuna chiamata all'azione oltre una: leggete il codice.

Esiste una forma di dissenso che consiste nel chiedere a chi comanda di comportarsi diversamente. Ne esiste un'altra che consiste nel costruire qualcosa che non ne abbia bisogno. Questo progetto appartiene alla seconda. Non perché la prima sia illegittima, ma perché un'alternativa funzionante è un argomento più duraturo di una protesta — e perché un sistema che elimina il bisogno di fiducia non può essere convinto a cambiare idea.

Il resto di questo documento descrive come, e — altrettanto importante — dove sono i limiti.

4. Da DMX a DMN

4.1 Cosa c'era prima

Daimon non è nato come protocollo decentralizzato. È nato come DMX (`0x36EbA94407B53c631eE822C219e94580fadd67c7`), un reflection token su BNB Chain con la struttura comune a quella categoria: un account proprietario con diciassette funzioni amministrative, e una fee dell'11% sulle transazioni.

	DMX (precedente)
Fee reflection	4%
Fee marketing	5%
Fee buyback	2%
Fee totale	11%
Tetto massimo fee	nessuno
Proprietario	sì, ownership mai rinunciata
Chi decide	il proprietario, con effetto immediato
Floor di supply	nessuno
Riduzione della supply	nessuna — vedi 4.1.2
Staking	nessuno
Governance	nessuna

Questa sezione non è scritta come un'accusa. DMX è il nostro lavoro precedente, e la struttura descritta qui era il pattern standard della sua generazione. È documentata precisamente perché la community che possiede quei token merita di sapere esattamente da cosa sta migrando — e perché diverse di queste proprietà non sono visibili senza leggere il sorgente.

4.1.1 Parametri senza limiti

I valori delle fee non erano fissati nel codice. Erano parametri del costruttore, modificabili in seguito da tre funzioni setter:

```
function setTaxFee(uint256 taxFee) external onlyOwner() {
    _taxFee = taxFee;
}
```

Un'assegnazione secca. Nessun `require`, nessun limite superiore, nessun ritardo, nessun annuncio. Il proprietario poteva impostare qualsiasi fee a qualsiasi valore, incluso il 100%, con effetto dalla transazione successiva.

Lo stesso vale per la dimensione massima delle transazioni. Deployata allo 0,3% della supply, oggi è allo 0,15% — il proprietario l'ha dimezzata a un certo punto usando un setter senza limite inferiore. Nulla nel codice impediva di portarla a un wei, il che avrebbe reso il token di fatto intrasferibile.

4.1.2 Il burn che non bruciava

DMX non aveva una funzione di burn. Aveva un buyback che comprava token sul mercato e li inviava all'indirizzo morto `0x...dEaD`.

La distinzione non è semantica. I token inviati a un indirizzo non spendibile sono economicamente distrutti ma restano nella contabilità:

	DMX, on-chain (26 luglio 2026)
<code>totalSupply()</code>	1.000.000.000.000 — invariata dal deploy
Detenuti dall'indirizzo morto	30.058.718.442 (3,0058%)
Circolante	969.941.281.557

Il saldo dell'indirizzo morto continua a crescere, perché il meccanismo di buyback è tuttora attivo; la cifra sopra è una fotografia. La supply totale non è una fotografia — non è mai diminuita di un solo wei. Qualsiasi metrica derivata

da `totalSupply()` — la capitalizzazione di mercato tra queste — era calcolata su una cifra che sovrastimava i token esistenti.

Non è una peculiarità di DMX; è il modo in cui la maggior parte dei reflection token di quella generazione gestiva il burn. DMN separa i due passaggi: i token vengono inviati all'indirizzo morto, e una seconda funzione `permissionless` rimuove davvero quel saldo da `totalSupply()`, limitata dal floor.

4.1.3 Rinuncia all'ownership, reversibile

Il contratto include una funzione che vale la pena descrivere per intero, perché il suo aspetto e il suo effetto divergono:

```
function lock(uint256 time) public virtual onlyOwner {
    _previousOwner = _owner;
    _owner = address(0);           // sembra rinunciata
    _lockTime = block.timestamp + time;
}

function unlock() public virtual {
    require(_previousOwner == msg.sender, ...);
    require(block.timestamp > _lockTime, ...);
    _owner = _previousOwner;       // se la riprende
}
```

Dopo aver chiamato `lock()`, `owner()` restituisce l'indirizzo zero. Su qualsiasi block explorer questo è indistinguibile da una rinuncia permanente — il segnale più comunemente citato come prova che un progetto non possa essere alterato dai suoi creatori. È temporaneo, e il proprietario precedente può riprendersi il controllo alla scadenza del timer.

Questa funzione non è mai stata usata su DMX (`getUnlockTime()` restituisce zero, e l'ownership non è mai stata rinunciata). È documentata qui perché il pattern è diffuso, e perché chi valuta un qualsiasi token dovrebbe sapere che `owner() == 0x0` non è, di per sé, prova di nulla.

4.1.4 Cosa DMX non aveva — detto con equità

Diversi poteri comunemente presenti in token di questo tipo sono assenti da DMX, e la precisione richiede di dirlo:

- **Nessuna funzione di emissione.** Nessuna. La supply era fissata al deploy e non poteva essere espansa da nessuno. Su questo punto il contratto era già solido.
- **Nessuna pausa e nessuna blacklist.** Il proprietario non poteva congelare i trasferimenti né bloccare indirizzi specifici.
- **Nessuna funzione di prelievo o recupero.** Il proprietario non poteva estrarre fondi arbitrari detenuti dal contratto.

Il problema di DMX non è mai stato che potesse essere svuotato. Era che la sua economia dipendeva interamente dalla discrezione di un account, senza limiti, senza ritardo, e senza obbligo di informare nessuno.

Un ultimo dettaglio completa il quadro: l'indirizzo che riceve la fee marketing è lo stesso indirizzo che possiede il contratto. Chi controlla i parametri è anche il destinatario diretto delle entrate che controlla. Nulla di questo era nascosto — è visibile on-chain a chiunque guardi — ma è esattamente la concentrazione che la nuova architettura è progettata per eliminare.

4.1.5 Su come si è arrivati a questo

DMX è stato il nostro apprendistato. L'abbiamo costruito con le conoscenze che avevamo allora, seguendo i pattern che erano standard per quella generazione di token — pattern condivisi da centinaia di progetti, inclusi molti con team finanziati e audit pubblicati.

Quello che è seguito è stato un periodo di studio, e questo protocollo è ciò che ne è uscito. La migrazione esiste perché la risposta onesta a comprendere un limite non è difenderlo.

Vale la pena notare cosa DMX *non* ha ereditato da quei template: nessuna funzione di emissione, nessuna pausa, nessuna blacklist, nessun meccanismo di prelievo. Le quattro capacità più spesso usate per estrarre valore da chi detiene erano assenti già dalla prima versione. Il giudizio c'era prima del vocabolario.

La nuova versione non tenta di difendere quella struttura. La sostituisce con una migliore.

4.2 Cosa la sostituisce

	DMX (precedente)	DMN (nuovo)
Fee reflection	4%	1%
Fee marketing	5%	2% — di cui il 60% va agli staker
Fee buyback	2%	1%
Fee totale	11%	4%
Tetto massimo fee	nessuno	10%, imposto nel codice
Proprietario	sì, mai rinunciato	nessuno, strutturalmente
Chi decide	il proprietario, subito	voto on-chain + timelock 7 giorni
Modifica parametri	istantanea, non annunciata	13 giorni minimo, interamente pubblica
Emissione	non possibile	non possibile
Riduzione supply	token spostati, supply invariata	supply realmente ridotta
Floor di supply	nessuno	21B, immutabile
Esclusione da reflection	modificabile dal proprietario	insieme immutabile, solo dead address
Protezione slippage	nessuna — accetta qualsiasi output	limitata, con quote dal router
Destinatario marketing	stesso indirizzo del proprietario	impostato dalla governance, previsto multisig
Staking	nessuno	vote-escrow, reward in BNB
Governance	nessuna	governance on-chain completa

Tre punti meritano enfasi.

La riduzione delle fee non è la parte importante. Le fee possono essere modificate da qualsiasi progetto in qualsiasi momento; un numero più basso non dimostra nulla su come ci si sia arrivati. Ciò che conta è che il 4% di DMN è stato stabilito da un voto on-chain, eseguito dopo un timelock pubblico di sette giorni, e che nessun singolo account può cambiarlo di nuovo. Il registro completo di quella decisione — proposta, voto, attesa, esecuzione, con gli hash delle transazioni — è documentato nella Sezione 9.

Il tetto conta più del valore attuale. Il codice rifiuta qualsiasi fee totale superiore al 10%. Non è una politica che la governance possa rivedere: è un'istruzione `require` nel contratto. Anche se ogni possessore di token votasse a favore, la transazione fallirebbe. La conseguenza pratica è che DMN non potrà mai diventare costoso da scambiare quanto DMX lo è oggi, indipendentemente da chi controllerà il protocollo tra dieci anni.

La riga sulla riduzione della supply è la meno ovvia e la più consequenziale. In DMX i token del buyback si accumulavano all'indirizzo morto mentre `totalSupply()` restava per sempre al valore di deploy. In DMN una funzione `permissionless` rimuove davvero quel saldo dal totale — fino al floor e non oltre. La supply riportata è la supply.

4.3 Come funziona la migrazione

La migrazione è uno scambio 1:1 senza fee e senza pressione temporale oltre una finestra prefissata.

Al deploy, l'intera supply iniziale di 1.000 miliardi di DMN viene creata dentro il contratto di migrazione. Non in un wallet del team, non in una tesoreria, non in un contratto di distribuzione — dentro il contratto che esiste unicamente per scambiare vecchi token con nuovi. Nessuna allocazione è riservata a fondatori, advisor o investitori privati, perché non esiste alcuna allocazione da riservare.

Il processo per chi detiene è in tre passaggi:

1. **Approvazione** — si autorizza il contratto di migrazione a ricevere una quantità specificata di token legacy.
2. **Claim** — il contratto prende i token legacy e restituisce esattamente lo stesso numero di DMN. Nessuna fee viene applicata in nessuna delle due direzioni.
3. **Fatto** — i token legacy vengono trasferiti alla tesoreria della DAO, non distrutti.

Ogni claim è un trasferimento di token che esistevano già. Nulla viene emesso. La supply totale di DMN non cambia di un solo wei durante l'intera migrazione, e questo è verificabile on-chain in qualsiasi momento.

4.4 Cosa succede ai token mai riscattati

Non tutti migreranno. I wallet vengono abbandonati, le chiavi si perdono, gli annunci non vengono letti. Una porzione della supply resterà nel contratto di migrazione alla chiusura della finestra.

Quei token non vengono bruciati automaticamente, e non diventano proprietà di nessuno. Restano bloccati nel contratto di migrazione finché la DAO non decide diversamente — e quella decisione richiede il ciclo di governance completo: una proposta, un voto, un timelock di sette giorni, e un'esecuzione che chiunque può innescare.

La funzione che esegue questo trasferimento, `sweepUnclaimed()`, ha tre vincoli scritti dentro di sé:

- può essere chiamata solo dal timelock, ovvero solo come risultato di un voto approvato;
- fallisce se chiamata prima che la scadenza della migrazione sia passata;
- può inviare i token a un solo indirizzo — la tesoreria — il cui indirizzo è `immutable`, fissato al deploy e imm modificabile da chiunque, governance inclusa.

C'è un dettaglio che vale la pena dichiarare apertamente, perché altrimenti verrebbe scoperto e contestato: il contratto di migrazione è un possessore di token come gli altri, e quindi accumula reflection dai normali trasferimenti durante la finestra di migrazione. Il saldo trasferito alla tesoreria sarà leggermente superiore alla quantità non riscattata. È il meccanismo di reflection che funziona come progettato, e il surplus segue lo stesso percorso del resto — alla DAO, deciso dal voto.

4.5 Perché una migrazione invece di rinunciare all'ownership

Una domanda ragionevole: se l'obiettivo era rimuovere il proprietario, perché non chiamare semplicemente `renounceOwnership()` sul contratto esistente?

Perché rinunciare all'ownership su DMX avrebbe prodotto un token che nessuno poteva cambiare — incluso un token che nessuno poteva correggere, migliorare o governare. La fee sarebbe rimasta congelata all'11% per sempre. Non ci sarebbe stato staking, né voto, né alcun modo per chi detiene di decidere alcunché. Il risultato non è decentralizzazione; è abbandono.

Decentralizzazione significa che le decisioni vengono prese collettivamente, non che le decisioni diventano impossibili. Questo richiede un sistema di governance, un timelock, un meccanismo di staking per pesare la partecipazione, e contratti progettati attorno all'assenza di un amministratore dalla prima riga. Nulla di tutto ciò poteva essere aggiunto a posteriori al contratto esistente.

La migrazione è il costo di farlo come si deve.

5. Architettura

5.1 Cinque contratti

Il protocollo è composto da cinque contratti, ciascuno con una responsabilità circoscritta.

DaimonV2 — il token. Implementa l'interfaccia ERC-20 con redistribuzione delle fee basata su reflection, il ciclo di buyback e burn, e il floor immutabile di supply. Deployato dietro un proxy UUPS, aggiornabile solo attraverso la governance.

DaimonStaking — lo staking vote-escrow. Detiene i token bloccati, calcola il potere di voto in funzione di quantità e durata del lock, mantiene i checkpoint storici di quel potere, e distribuisce i reward in BNB in modo proporzionale.

DaimonGovernor — il motore di governance. Accetta proposte da chi detiene oltre una soglia, gestisce il periodo di voto, valuta il quorum contro uno snapshot preso alla creazione della proposta, e mette in coda nel timelock le proposte approvate.

DaimonTimelock — il ritardo obbligatorio. Trattiene ogni decisione approvata per sette giorni prima che possa essere eseguita, ed è l'unico account con l'autorità di chiamare le funzioni privilegiate sugli altri contratti. Si amministra da sé: nessun account esterno può modificarne i parametri o i ruoli.

DaimonMigration — lo scambio 1:1. Detiene la supply iniziale, converte i token legacy su richiesta, e può trasferire l'eventuale residuo non riscattato alla tesoreria dopo la scadenza, solo su istruzione del timelock.

5.2 Dove sta l'autorità

La distribuzione dell'autorità è la proprietà che più vale la pena verificare in modo indipendente, quindi è dichiarata qui per intero.

Nessun contratto ha un proprietario. Non c'è ereditarietà da `Ownable`, non c'è `owner()`, non c'è alcun modificatore `onlyOwner` in nessun punto del protocollo. Il controllo degli accessi è basato su ruoli, e dopo il deploy esiste esattamente un detentore di ruolo:

```
GOVERNANCE_ROLE → detenuto da DaimonTimelock, e da nient'altro
```

Chi ha fatto il deploy non detiene alcun ruolo. Non è una questione di intenzioni: lo script di deploy rinuncia a ogni ruolo che detiene temporaneamente durante la configurazione, e poi verifica — con quattordici controlli separati che interrompono il deploy se anche uno solo fallisce — che nessun account esterno mantenga autorità su alcun contratto. Se un solo controllo fallisce, il deploy non avviene.

Di conseguenza, cambiare le fee, cambiare il wallet marketing, cambiare il contratto di staking, aggiungere un'opzione di lock, trasferire i token di migrazione non riscattati, o aggiornare l'implementazione del token sono tutte operazioni che possono avvenire solo al termine di questa sequenza:

```
proposta → 1 giorno di attesa → 5 giorni di voto → quorum raggiunto  
→ coda → 7 giorni di timelock → esecuzione
```

L'esecuzione stessa è permissionless: una volta trascorso il timelock, qualsiasi indirizzo può innescarla. La decisione l'ha presa il voto; l'esecuzione è meccanica.

5.3 Le due eccezioni

Due elementi si collocano fuori da questa struttura, ed entrambi sono deliberati.

Indirizzi immutabili. L'indirizzo morto (destinazione dei token bruciati) e la tesoreria della migrazione sono fissati al deploy e non possono essere cambiati da nessuno — né da chi ha fatto il deploy, né dalla governance, né da un aggiornamento. Alcune destinazioni non dovrebbero essere reindirizzabili, nemmeno da una maggioranza.

Il guardian. Un account detiene un potere: può mettere in pausa il contratto in caso di emergenza. Non può cambiare le fee, muovere fondi, alterare la governance, emettere token o aggiornare alcunché. Esiste perché nella prima fase di vita di un protocollo sette giorni di timelock sono una risposta troppo lenta a un exploit in corso.

L'autorità del guardian scade 36 mesi dopo il deploy. Non è un impegno a rinunciarvi; è un confronto tra timestamp nel codice che nessuno può rimuovere o posticipare. Dopo quella data la funzione di pausa smette di funzionare permanentemente — mentre la funzione di riattivazione continua a funzionare, così che nessuno possa lasciare il protocollo congelato.

La Sezione 8.6 descrive i vincoli del guardian per intero.

6. Tokenomics

6.1 Supply

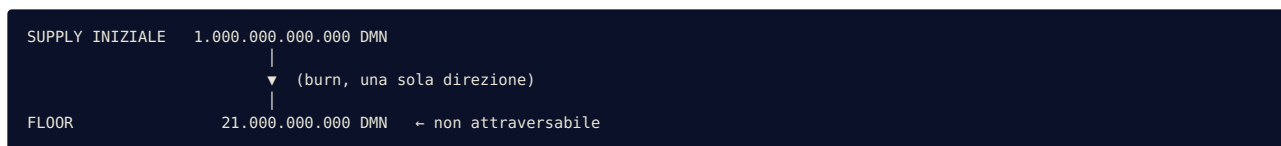
La supply iniziale è di 1.000.000.000.000 DMN — mille miliardi di token, con 18 decimali. Questo numero è stato fissato al deploy e non può mai aumentare.

La ragione per cui non può mai aumentare non è una politica. **Non esiste una funzione di emissione nel contratto.** Non disattivata, non protetta da permessi, non riservata alla governance: assente. Nessuna sequenza di voti, nessun aggiornamento, nessuna chiave compromessa e nessuna decisione futura può produrre un singolo token aggiuntivo, perché il codice in grado di produrlo non esiste.

La supply può muoversi in una sola direzione: verso il basso.

6.2 Il floor

Il burn è limitato. Il contratto definisce una supply minima immutabile di 21.000.000.000 DMN — ventuno miliardi — sotto la quale bruciare è matematicamente impossibile.



La funzione di burn verifica questo limite prima di eseguire e si ferma esattamente al floor: se un burn portasse la supply sotto i 21 miliardi, viene bruciata solo la porzione che raggiunge il floor, e il resto no. Chiamata nuovamente in seguito, la funzione non esegue alcuna operazione.

Questo tetto alla deflazione esiste per una ragione pratica. Una supply che può avvicinarsi a zero prima o poi si rompe: i limiti di divisibilità, il comportamento degli arrotondamenti e la liquidità degradano tutti man mano che il conteggio dei token collassa. Il floor è il punto in cui il protocollo smette di ridursi e inizia a operare in una modalità diversa — descritta in 6.6.

6.3 Fee sulle transazioni

Ogni trasferimento applica una fee, attualmente il 4% dell'importo trasferito, suddivisa in tre componenti:

Componente	Quota	Destinazione
Reflection	1%	ridistribuita a tutti i possessori
Buyback & burn	1%	accumulata, poi usata per comprare e bruciare
Marketing / operativo	2%	60% ai reward degli staker, 40% alle operazioni
Totale	4%	

Due proprietà strutturali contano più dei numeri attuali.

Il tetto. Il contratto rifiuta qualsiasi configurazione in cui la fee totale superi il 10%. È imposto da un'istruzione `require`, non da una politica. Una proposta che impostasse le fee all'11% supererebbe il voto, aspetterebbe il timelock, e poi fallirebbe all'esecuzione. La governance è limitata dal codice che governa.

Chi può cambiarle. Solo il timelock, ovvero solo l'esito di un ciclo di governance completato. L'attuale 4% è esso stesso il risultato di uno: la prima proposta nella storia di Daimon ha ridotto le fee dal 5% al 4%, e il suo registro completo compare nella Sezione 9.

6.4 Reflection

La componente reflection ridistribuisce l'1% di ogni trasferimento a tutti i possessori, proporzionalmente al loro saldo, senza che sia necessaria alcuna transazione per riscuoterla.

Meccanicamente, i saldi sono memorizzati in un'unità di conto interna invece che direttamente in token. Ogni trasferimento tassato riduce il totale di quell'unità interna, il che aumenta il numero di token che ciascuna unità rappresenta. Il saldo di ogni possessore cresce senza che avvenga alcun trasferimento — la redistribuzione è un cambiamento nel tasso di conversione, applicato simultaneamente a tutti.

La conseguenza pratica è che detenere token produce un lento accumulo finanziato dall'attività di scambio, senza alcuna azione richiesta e senza pagare gas.

Un indirizzo è escluso dalla reflection: l'indirizzo morto che riceve i token bruciati. Escluderlo impedisce che la supply bruciata accumuli redistribuzione, il che gonfierebbe la cifra apparente del burn. I token contati come bruciati sono bruciati; nulla viene aggiunto a quel numero dal meccanismo stesso.

Non esistono altre esclusioni, e non esiste alcuna funzione per crearne. È una scelta strutturale deliberata: la capacità di inserire e togliere indirizzi dalla reflection a runtime è stata storicamente la fonte dei più gravi fallimenti contabili nei reflection token. Quell'intera classe di fallimenti è assente qui perché la capacità è assente.

6.5 Il ciclo di burn

Il burn non è programmato, annunciato, o innescato da qualcuno in particolare. È una conseguenza dell'attività di scambio.

1. Avvengono scambi → le fee si accumulano come DMN nel contratto
2. Quando le fee accumulate superano una soglia, e avviene una vendita verso la pool di liquidità, il contratto vende i token accumulati sul mercato → riceve BNB
3. Il BNB viene suddiviso:
 - quota marketing → 60% al pool reward dello staking
 - 40% alle operazioni
 - quota buyback → trattenuta nel contratto
4. Quando il BNB trattenuto supera una soglia, il contratto compra DMN sul mercato aperto e li invia all'indirizzo morto
5. `burnDeadBalanceToFloor()` rimuove quel saldo dalla supply totale — permanentemente, e al massimo fino al floor

Il passaggio 5 è **permissionless**: qualsiasi indirizzo può chiamarlo, in qualsiasi momento, senza alcun ruolo speciale. Nessuno può impedirlo, e nessuno può forzarlo oltre il floor. La funzione non ha un chiamante privilegiato perché non ne ha bisogno — il suo unico effetto possibile è ridurre la supply verso un limite fissato nel codice.

I passaggi 2 e 4 sono automatici e usano la pool di liquidità pubblica. Gli swap sono protetti da un limite di slippage e incapsulati in modo che uno swap fallito non possa bloccare i trasferimenti: se le condizioni di mercato rendono uno swap sfavorevole, viene saltato e ritentato più tardi invece di far fallire la transazione dell'utente.

6.6 Cosa succede al floor

Quando la supply totale raggiunge i 21 miliardi, il burn si ferma permanentemente. Non riprende, e nessun voto può riavviarlo.

A quel punto la componente buyback non ha più una destinazione che riduca la supply. Il progetto del protocollo specifica che le entrate precedentemente destinate al burn vengono reindirizzate interamente agli staker — il meccanismo passa dal ridurre la supply al distribuire rendimento, in modo automatico, sulla base di un controllo sulla supply e non di una decisione.

È uno scenario lontano. È descritto qui perché un protocollo dovrebbe specificare il proprio stato terminale, non perché sia imminente.

7. Staking

7.1 Lock e peso

Lo staking in Daimon è vote-escrow: i token vengono bloccati per una durata scelta, ed entrambi — potere di voto e quota di reward — sono pesati da quella durata.

Periodo di lock	Moltiplicatore
30 giorni	1×
90 giorni	1,5×
180 giorni	2,2×
365 giorni	4×

Il potere di voto è l'importo bloccato moltiplicato per il peso del periodo. Un milione di DMN bloccati per un anno pesa quanto quattro milioni bloccati per un mese.

L'intenzione è esplicita: **l'influenza dovrebbe seguire l'impegno, non solo il capitale**. Chi accetta un anno di illiquidità ha dimostrabilmente più in gioco nel futuro del protocollo di chi può uscire in trenta giorni, e la pesatura lo riflette.

I token bloccati non possono essere ritirati prima della scadenza del lock. Non esiste uscita anticipata, né opzione con penale, né override amministrativo. L'impegno è il meccanismo; renderlo reversibile ne cancellerebbe il significato.

7.2 Reward in BNB

I reward dello staking sono pagati in **BNB**, mai in DMN.

È una conseguenza del design della supply più che una preferenza. Senza funzione di emissione, reward denominati in DMN potrebbero provenire soltanto da una riserva pre-allocata — che prima o poi si esaurirebbe, e che avrebbe richiesto di detenere un grosso saldo controllato dal team fin dal primo giorno. Nessuna delle due cose era accettabile.

I reward provengono quindi dalla quota marketing delle fee sulle transazioni: il 60% di quella quota viene convertito in BNB dal protocollo e depositato nel pool reward dello staking, dove viene distribuito tra gli staker in proporzione al potere di voto.

Ne derivano tre proprietà:

- **Nessuna inflazione.** Nessun token viene creato per pagare nessuno. Il pool reward è finanziato da attività di scambio già avvenuta.
- **Nessuna pressione di vendita.** Uno staker che riscuote riceve BNB, non DMN. Riscuotere non mette mai token sul mercato.
- **Valore esterno.** I reward sono denominati nell'asset nativo della chain, il cui valore non dipende da Daimon.

L'effetto combinato è che lo staking rimuove token dalla circolazione restituendo valore che non deve essere rivenduto sullo stesso mercato.

I reward maturano continuamente e si riscuotono su richiesta. Se dei reward arrivano in un momento in cui nessun token è in staking, vengono trattenuti e distribuiti agli staker successivi invece di andare persi.

7.3 Checkpoint del potere di voto

Il contratto di staking non registra solo il potere di voto attuale. Ne registra la storia.

Ogni cambiamento — un nuovo lock, un ritiro — scrive un checkpoint con un timestamp. Questo permette al contratto di governance di porre una domanda specifica: *qual era il potere di voto di questo indirizzo nel momento in cui la proposta N è stata creata?*

È ciò che impedisce l'attacco alla governance più evidente. Senza checkpoint storici, un attore potrebbe osservare una proposta che non gli piace, acquisire e mettere in staking una posizione ingente, e votarla contro con potere acquistato dopo che la questione è stata sollevata. Con i checkpoint, il voto viene risolto contro uno snapshot preso alla creazione della proposta: il potere acquisito in seguito non conta nulla.

Lo abbiamo testato specificamente, con un attaccante simulato che deteneva una posizione schiacciante messa in staking subito dopo la creazione di una proposta. Il suo potere di voto allo snapshot era zero, e il voto è stato rifiutato — come progettato.

7.4 Una scelta di design: il potere di voto non decade

Nel modello vote-escrow introdotto da Curve Finance, il potere di voto decade linearmente man mano che il lock si avvicina alla scadenza. Un lock di quattro anni conferisce peso pieno il primo giorno e peso zero a maturità; mantenere influenza richiede di ri-bloccare continuamente.

Daimon non implementa il decadimento. Il potere di voto è fissato al momento dello staking e resta costante fino al ritiro, anche dopo la scadenza del lock.

È una divergenza deliberata, e ha una conseguenza reale che va dichiarata apertamente: chi blocca a 4×, aspetta la fine dell'anno e non ritira mai mantiene peso di voto pieno a tempo indefinito pur avendo la possibilità di uscire in qualsiasi momento. L'influenza può concentrarsi tra gli staker storici di lungo periodo invece di seguire l'impegno presente.

Accettiamo questo compromesso per tre ragioni. È più semplice e prevedibile — il potere di voto è un numero che chi detiene può comprendere senza ricalcolarlo a ogni blocco. Premia la fedeltà dimostrata invece di richiedere un re-impegno perpetuo. Ed evita l'attrito di un meccanismo che penalizza chi detiene proprio mentre si avvicina alla fine di un impegno che ha onorato.

Un modello con decadimento non è escluso per sempre. Sarebbe una riprogettazione significativa, e se la community concludesse che è preferibile, può essere introdotto attraverso lo stesso processo di governance che regola tutto il resto. La scelta è documentata qui invece di essere scoperta dopo.

7.5 Limiti

Due vincoli si applicano allo staking e sono dichiarati per completezza.

Una singola transazione di staking è limitata dalla dimensione massima di transazione del token (0,5% della supply),

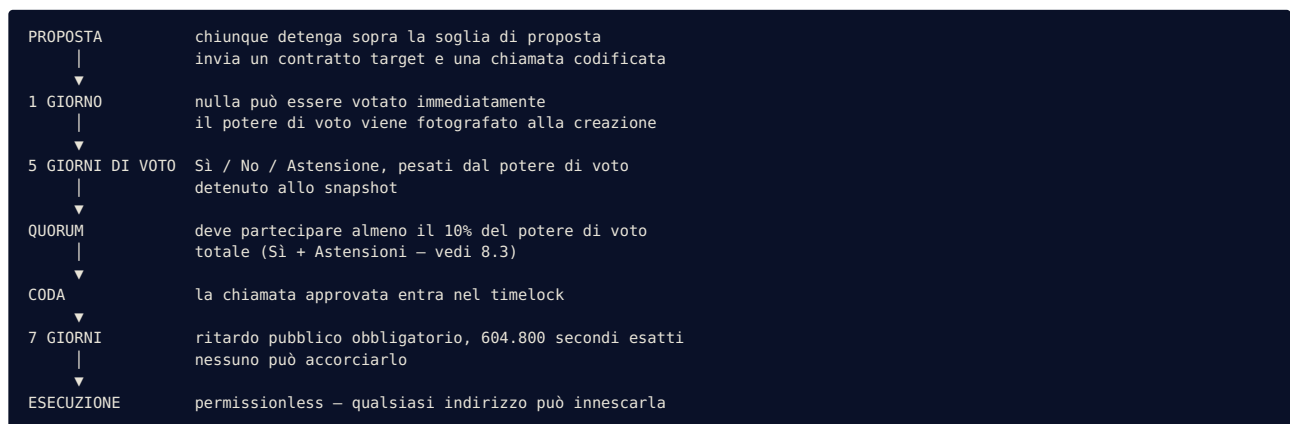
un limite anti-dump che si applica a tutti i trasferimenti. Posizioni molto grandi devono essere messe in staking in più transazioni.

Il contratto di staking è escluso automaticamente dalle fee quando viene registrato, quindi mettere e togliere dallo staking non comporta la fee del 4%. Questa esclusione si applica al contratto stesso, non a un individuo, ed è impostata dalla stessa funzione riservata alla governance che registra il contratto.

8. Governance

8.1 Il ciclo

Ogni modifica al protocollo segue lo stesso percorso. Non ci sono scorciatoie, non ci sono procedure d'emergenza che lo aggirino, e non ci sono account esenti.



Tempo minimo dalla proposta all’effetto: **tredici giorni**. Non è una comodità. È la finestra in cui chiunque — un possessore, un ricercatore, un giornalista, un avversario — può leggere cosa sta per accadere e agire prima che accada.

8.2 Il potere di voto allo snapshot

I voti sono pesati dal potere di voto detenuto **nel momento in cui la proposta è stata creata**, non nel momento del voto.

Questa singola proprietà chiude l’attacco più diretto a qualsiasi governance pesata sui token: osservare una proposta, acquisire una posizione ingente, e votare con influenza acquistata specificamente per decidere quella questione. Con il voto su snapshot, i token messi in staking dopo che una proposta esiste hanno peso zero su di essa.

Il meccanismo è un checkpoint storico scritto a ogni modifica della posizione in staking di un possessore. Il contratto di governance interroga quella storia invece dello stato presente.

8.3 Cosa conta per il quorum

Il quorum è misurato come somma dei voti **a favore e di astensione**, contro il 10% del potere di voto totale allo snapshot. I voti contrari contano nel determinare l’esito, ma non nel raggiungere il quorum.

Questa distinzione non è estetica, e non era nella nostra implementazione originale. Durante i test avversariali abbiamo scoperto che conteggiare i voti contrari nel quorum produceva un incentivo perverso: una minoranza contraria a una proposta poteva, votando contro, fornire la partecipazione necessaria a validare il voto — e quindi far passare la proposta. Restare in silenzio era strettamente più efficace che votare no.

Abbiamo riprodotto lo scenario in un test, lo abbiamo confermato, e abbiamo cambiato il calcolo per allinearli allo standard OpenZeppelin, che esclude i voti contrari dal quorum esattamente per questa ragione. Il finding, la correzione e il test che dimostra il comportamento corretto sono tutti nel repository pubblico.

Una proposta che nessuno sostiene ora fallisce per mancato raggiungimento del quorum — che è l’esito corretto, e non richiede a nessuno di sconfiggerla attivamente.

8.4 Limiti alla governance stessa

Un sistema in cui la maggioranza può fare qualsiasi cosa non è più sicuro di un sistema in cui una persona può fare qualsiasi cosa; è solo più lento. Diversi vincoli sono quindi posti al di sopra della governance, nel codice, dove nessun voto può raggiungerli:

Vincolo	Valore	Chi può cambiarlo
Fee totale massima	10%	nessuno
Supply minima (floor)	21 miliardi	nessuno
Quorum minimo	10%	nessuno
Ritardo minimo del timelock	7 giorni	nessuno
Indirizzo morto	fissato al deploy	nessuno
Tesoreria della migrazione	fissata al deploy	nessuno
Capacità di emissione	non esiste	nessuno

Una proposta che violasse uno di questi supererebbe il voto, aspetterebbe il timelock, e fallirebbe all'esecuzione. Il protocollo rifiuta le istruzioni che è stato costruito per rifiutare.

8.5 Cosa può fare la governance

Entro quei limiti, la DAO controlla completamente il protocollo:

- modificare la ripartizione e il totale delle fee (fino al tetto)
- cambiare il destinatario marketing e operativo
- cambiare il contratto di staking e la ripartizione dei reward
- aggiungere o disabilitare opzioni di lock dello staking
- modificare le soglie operative (innesco degli swap, tolleranza allo slippage, dimensione massima delle transazioni)
- trasferire alla tesoreria i token di migrazione non riscattati dopo la scadenza
- **aggiornare l'implementazione del token stesso**

L'ultimo punto merita enfasi. Il token è deployato dietro un proxy UUPS, e la funzione di autorizzazione dell'aggiornamento è riservata al timelock. Questo significa che il protocollo può evolvere — ma solo attraverso il ciclo pubblico completo, e mai a discrezione di un individuo. Significa anche che la community porta una responsabilità reale: un aggiornamento è lo strumento più potente del sistema, e il ritardo di sette giorni esiste perché un cattivo possa essere visto arrivare.

8.6 Il guardian

Un account detiene un potere fuori dal ciclo di governance. Il guardian può **mettere in pausa il contratto**. Questo è l'intero raggio della sua autorità.

Non può cambiare le fee, muovere fondi, alterare i parametri di governance, emettere token, aggiornare, o influenzare un voto. Può fermare i trasferimenti, e nient'altro.

Esiste per una ragione precisa: nella prima fase di vita di un protocollo, sette giorni di timelock non sono un tempo di risposta praticabile a un exploit in corso. Il guardian è l'allarme antincendio, non un posto al tavolo.

Tre vincoli lo definiscono:

Scade. L'autorità del guardian termina 36 mesi dopo il deploy. È un confronto tra timestamp nel codice, non un impegno — nessuno, governance inclusa, può rimuoverlo o prorogarlo. Dopo quella data la funzione di pausa smette di funzionare permanentemente.

Non può intrappolare il protocollo. La funzione di riattivazione continua a funzionare anche dopo la scadenza del guardian. Il potere di congelare scompare; quello di ripartire no. Nessuno può lasciare Daimon in pausa a tempo indefinito.

È visibile. Mettere in pausa è un evento on-chain. Non c'è modo di usare questo potere in silenzio.

Il guardian è un compromesso temporaneo con la realtà, limitato nel raggio, limitato nel tempo, e progettato per scomparire senza richiedere la collaborazione di nessuno.

9. Caso di studio: Proposta #0

Ogni affermazione della Sezione 8 descrive un comportamento previsto. Questa sezione descrive cosa è realmente accaduto quando il sistema è stato usato.

Quello che segue è il registro completo della prima decisione presa dalla DAO di Daimon, eseguita su BSC testnet. Ogni passaggio è una transazione pubblica.

9.1 La decisione

Proposta #0 — ridurre le fee totali dal 5% al 4% (reflection 1%, buyback 1%, marketing 2%).

La proposta è stata creata da un possessore, ha come target il contratto del token, e codifica una chiamata a `setFees(10, 10, 20)`. Nulla nella sua creazione ha richiesto un permesso.

La sua descrizione on-chain recita:

“Riduzione fee totale al 4% (tax 1%, buyback 1%, marketing 2%)”

Il testo è in italiano e compare non tradotto in tutta l’interfaccia, inclusa la versione inglese. Le descrizioni delle proposte sono contenuto on-chain scritto dal proponente e immutabile una volta inviato; tradurle nell’interfaccia romperebbe la corrispondenza tra ciò che un utente legge e ciò che la blockchain contiene. Ciò che viene mostrato è byte per byte ciò che è stato registrato.

9.2 La cronologia

Tutti gli orari in UTC. Ogni valore qui sotto è letto dallo stato del contratto, non dalla documentazione di progetto.

Fase	Timestamp	UTC
Creata — potere di voto fotografato	1783467501	7 lug 2026, 23:38:21
Voto aperto (dopo 1 giorno di attesa)	1783553901	8 lug 2026, 23:38:21
Voto espresso — 3.000.000 a favore		9 lug 2026, 22:36:42
Voto chiuso (dopo 5 giorni)	1783985901	13 lug 2026, 23:38:21
Messa in coda nel timelock	1783991100	14 lug 2026, 01:05:00
Eseguibile da	1784595900	21 lug 2026, 01:05:00
Eseguita		21 lug 2026, 01:11:09

Risultato: 3.000.000 di potere di voto a favore, 0 contrari, 0 astenuti, contro un quorum richiesto di 300.000. Stato finale: **Eseguita**.

L’intervallo tra la messa in coda e l’eseguibilità è di 604.800 secondi — sette giorni, al secondo.

Transazioni:

Azione	Hash
Proposta	0xa6e465fb70da2b587f8ab7795a22cfc7c29bc984d571020260178b6af2cb5035
Voto	0x90e22db141f83b1b9fb004faaabe852da721b55856594122c1f0cf9564e1480
Coda	0x3d5adeac84a205edd963af483de98bd0f40be0491532ed4dc9b0f2418fee2920
Esecuzione	0x5aa519a9884d24037f0cb903f3565f1a9e5e87529e5d4c1baa3f0c054302fe5f

Il ciclo completo è pubblico. Nulla di questa decisione è avvenuto fuori dalla catena.

Tutte e quattro sono state inviate dallo stesso indirizzo. Le transazioni di proposta, voto e coda occupano i nonce 6, 8 e 9 nella sequenza di quel wallet — l’ordine che il protocollo richiede, con il voto correttamente collocato tra le altre due.

9.3 La parte che conta di più

Il 17 luglio — quattro giorni prima che l’esecuzione fosse possibile — abbiamo tentato di eseguire la proposta.

È stato un errore umano: uno sbaglio sulla data, commesso da persone che avevano costruito il sistema e ne

conoscevano precisamente il funzionamento. Abbiamo eseguito un controllo di coerenza prima di firmare, e ha restituito la ragione per cui il tentativo sarebbe fallito:

```
readyTimestamp 21 lug 2026, 01:05:00 UTC
ora attuale    17 lug 2026, 08:34:44 UTC
mancano        318.613 secondi (3 giorni, 16,5 ore)
```

Se avessimo proceduto comunque, la transazione sarebbe fallita con `TooEarly`. Il contratto non distingue tra un'esecuzione anticipata malevola e una confusa; le rifiuta identicamente.

Questo è l'intero argomento a favore di un timelock, dimostrato invece che affermato. Non esiste perché diffidiamo delle cattive intenzioni. Esiste perché le buone intenzioni non bastano — le persone perdono il filo delle date, lavorano alle tre di notte, e sono certe di cose sbagliate. Il sistema no.

Quattro giorni dopo, l'esecuzione è riuscita sei minuti dopo l'apertura della finestra. Non un secondo prima, perché non era possibile.

9.4 Verifica del risultato

Il cambiamento delle fee è stato confermato su due livelli indipendenti.

Stato del contratto, letto direttamente:

```
taxFee      = 10 → 1% (reflection)
buybackFee  = 10 → 1% (buyback & burn)
marketingFee = 20 → 2% (marketing / operativo)
           _____
                    4%
```

Comportamento economico, misurato su un trasferimento reale di 1.000.000 DMN:

	Importo
Inviati	1.000.000,000000 DMN
Fee applicata	40.000,000000 DMN — esattamente il 4,00%
Ricevuti	960.000,000000 DMN
Variazione della supply totale	zero

Prima dell'esecuzione lo stesso trasferimento avrebbe comportato una fee di 50.000 DMN. Il ciclo di governance non ha semplicemente aggiornato un valore memorizzato: ha cambiato il modo in cui il token si comporta, ed è misurabile al wei.

9.5 La proposta non può essere eseguita due volte

Chiamare `execute(0)` di nuovo restituisce:

```
execution reverted: 0x0dc10197 → AlreadyExecuted()
```

Il controllo è la prima istruzione della funzione, prima di qualsiasi altra logica. Una transazione firmata e pagata fallirebbe identicamente. L'interfaccia non offre alcun pulsante per farlo, perché lo stato "eseguita" non ha azioni disponibili.

9.6 Una proposta fallita

La proposta #1 è stata creata deliberatamente come test e deliberatamente non votata da nessuno.

Al termine del suo periodo di voto di cinque giorni aveva zero voti, non ha raggiunto il quorum, e il suo stato è diventato **Bocciata** — automaticamente, senza alcuna transazione richiesta e senza che nessuno agisse per respingerla.

È l'esito che volevamo osservare on-chain. In un sistema senza potere di veto, una proposta senza sostegno non ha bisogno di essere fermata. Scade.

9.7 Cosa dimostra questo registro

Tra queste due proposte, ogni stato della macchina di governance è stato osservato con transazioni reali: In attesa, In votazione, Approvata, In coda, Eseguita, e Bocciata. I percorsi di rifiuto — esecuzione prima della scadenza del timelock, ed esecuzione di una proposta già eseguita — sono stati entrambi innescati ed entrambi respinti.

Il registro completo, inclusi gli hash di ogni passaggio, è pubblicato in `TESTNET_RESULTS.md` nel repository del progetto.

10. Sicurezza

10.1 Modello di minaccia

La sicurezza comincia dal nominare gli avversari. Sono considerati i seguenti attori, con ciò che ciascuno può e non può fare.

Un attaccante esterno. Può chiamare qualsiasi funzione pubblica, in qualsiasi ordine, in qualsiasi momento, con qualsiasi importo. Non può acquisire un ruolo amministrativo, perché nessuno è assegnabile: `GOVERNANCE_ROLE` amministra sé stesso e non esiste un `DEFAULT_ADMIN_ROLE` attraverso cui potrebbe essere concesso.

Una whale. Può accumulare una posizione ingente e metterla in staking per il peso massimo. Non può votare su alcuna proposta creata prima del suo staking, perché il potere di voto è valutato contro uno snapshot storico. Non può superare la dimensione massima di transazione in un singolo trasferimento.

La governance stessa. Può cambiare ogni parametro operativo e aggiornare l'implementazione del token. Non può emettere, non può bruciare sotto il floor, non può alzare le fee sopra il 10%, non può abbassare il quorum sotto il 10% o il timelock sotto i sette giorni, non può reindirizzare l'indirizzo morto o la tesoreria della migrazione. Il protocollo limita la propria stessa governance.

Il guardian. Può mettere in pausa il contratto. Non può fare nient'altro, e perde anche quello dopo 36 mesi.

Chi ha fatto il deploy. Può deployare i contratti e pagare il gas. Non detiene alcun ruolo in seguito: lo script di deploy rinuncia a ogni permesso temporaneo e poi verifica, con quattordici asserzioni che interrompono il deploy in caso di fallimento, che nessun account esterno mantenga autorità in alcun punto del sistema.

Il modello di minaccia completo, incluso il ragionamento dietro ogni conclusione, è pubblicato come `THREAT_MODEL.md` nel repository.

10.2 Cosa è stato testato

Il protocollo porta 74 test automatici, tutti superati, in cinque categorie.

I test unitari coprono le singole funzioni e i loro limiti.

I test di sequenza di governance verificano che nessun ordinamento di operazioni produca un'esecuzione senza un voto approvato, una coda completata e un timelock trascorso.

I test di fuzzing eseguono ogni proprietà contro 512 input casuali: che i trasferimenti non creino mai token, che lo staking conceda esattamente il potere di voto pesato, che il ritiro restituisca il capitale 1:1, che la migrazione sia esattamente 1:1, che i reward non superino mai i BNB depositati, che le fee non possano mai essere impostate sopra il tetto.

I test di invarianza sono basati su handler: un driver martella il sistema con sequenze casuali di azioni — 16.384 chiamate per invariante — mentre le seguenti condizioni devono valere a ogni passo:

- la supply totale non supera mai quella iniziale e non scende mai sotto il floor
- il potere di voto totale eguaglia la somma dei lock attivi, ed eguaglia la somma dei poteri di voto individuali
- la contabilità della migrazione è conservata
- il saldo dei reward eguaglia i depositi meno le riscossioni
- nessun ruolo amministrativo è acquisibile da alcun attore

I test avversariali attaccano il sistema come farebbe un avversario: manipolazione dello snapshot, valori limite estremi (un wei, l'intera supply, il floor esatto, un secondo prima della scadenza del timelock), e incentivi di teoria dei giochi.

L'analisi statica viene eseguita con Slither. Ogni finding di severità alta e media è stato esaminato e documentato — con il relativo ragionamento — come falso positivo in questo contesto oppure come condizione già mitigata; la

classificazione è pubblicata in `THREAT_MODEL.md`. I suggerimenti di hardening a bassa severità sono stati applicati al codice.

10.3 Cosa abbiamo trovato noi stessi

Due finding sono emersi dal nostro giro avversariale, quando il codice era per il resto completo. Entrambi sono documentati qui perché una sezione sulla sicurezza che riporta solo successi non è una sezione sulla sicurezza.

Finding 1 — I voti contrari contavano nel quorum. Come descritto in 8.3, questo creava un incentivo perverso: opporsi a una proposta poteva aiutarla a passare. Lo abbiamo riprodotto in un test, poi abbiamo cambiato il calcolo del quorum per contare solo Sì + Astensioni, allineandolo allo standard OpenZeppelin. Corretto prima che lo scope dell'audit fosse congelato.

Finding 2 — Il potere di voto non decade dopo la scadenza del lock. Descritto in 7.4. È una conseguenza di design più che un difetto: l'influenza può concentrarsi tra gli staker storici di lungo periodo. Lo abbiamo accettato e documentato invece di correggerlo, perché un meccanismo di decadimento è una riprogettazione e non una correzione. Resta aperto a una futura decisione di governance.

10.4 Limiti noti

I seguenti sono proprietà del sistema che consideriamo accettabili ma che chi legge dovrebbe conoscere. Nessuna di esse è nascosta nel codice.

Esposizione residua al MEV. Gli swap automatici sono protetti da un limite di slippage, che riduce ma non elimina gli attacchi sandwich. Eliminare completamente l'esposizione richiederebbe un oracolo di prezzo ponderato nel tempo; il limite è il compromesso scelto.

Aggiornabilità. Il token può essere aggiornato dalla governance. È una capacità, e le capacità comportano rischi: un aggiornamento dannoso approvato da una maggioranza è possibile in linea di principio. Il timelock di sette giorni esiste perché una proposta simile sia visibile per una settimana prima di poter avere effetto.

Polvere di arrotondamento. I calcoli di reflection e reward usano divisioni intere, lasciando importi di pochi wei non attribuiti. Lo abbiamo misurato precisamente durante la distribuzione dei reward su testnet: su tre riscossioni, il residuo era di due wei, ciascuno attribuibile a una divisione con troncamento. È contabilizzato, non ignorato.

Dipendenza dal router. Gli swap automatici dipendono dal router PancakeSwap. Se quel contratto fosse compromesso o la pool di liquidità prosciugata, il meccanismo di swap degraderebbe. Gli swap sono incapsulati in modo che un fallimento non possa bloccare i trasferimenti ordinari.

Il percorso meno provato. La sequenza accumulo fee → swap in BNB → distribuzione → buyback automatico è stata esercitata una volta su testnet, in condizioni di laboratorio, su una pool con liquidità minima. Non ha mai girato sotto slippage reale, volume reale o condizioni avversariali, e non potrà farlo prima del mainnet. La consideriamo la superficie meno provata del protocollo e l'abbiamo segnalata come tale a ogni auditor che abbiamo contattato.

10.5 L'interfaccia non è il protocollo

Una domanda che merita una risposta esplicita: cosa succede se il sito viene compromesso?

L'interfaccia web è un frontend pubblico stateless. Non detiene chiavi, non custodisce fondi, non mantiene database né sessioni autenticate, e non firma mai nulla. Ogni interazione con la catena avviene nel browser dell'utente, con il wallet dell'utente, richiedendo la firma dell'utente.

L'interfaccia non è mai nel percorso di custodia né in quello di firma. La sua superficie d'attacco è quindi la **disponibilità, non i fondi** — un frontend compromesso o non disponibile non può muovere i token di nessuno, perché non ha mai avuto la capacità di muoverli.

Il caso peggiore realistico è che la pagina smetta di funzionare. I contratti restano pienamente utilizzabili attraverso un block explorer o uno strumento da riga di comando, come lo sono stati durante mesi di test prima che l'interfaccia esistesse.

È una conseguenza deliberata dell'architettura, non un caso fortunato: il protocollo sono i contratti, e l'interfaccia è una comodità posta davanti a essi. Qualsiasi cosa si possa fare attraverso l'interfaccia si può fare senza.

Lo stesso ragionamento vale per le dipendenze software dell'interfaccia. Le vulnerabilità note che le riguardano sono documentate in `SECURITY.md`, insieme a una valutazione di se le condizioni necessarie a sfruttarle esistano o meno in questa applicazione. I contratti stessi non hanno dipendenze di questo tipo a runtime.

10.6 Cosa è stato dimostrato on-chain

Oltre ai test automatici, i seguenti cicli funzionali sono stati eseguiti e documentati su BSC testnet, ciascuno con gli hash delle transazioni:

- migrazione 1:1, verificata al wei
- applicazione delle fee e distribuzione della reflection, verificate al wei
- staking vote-escrow con potere di voto pesato, e rifiuto del ritiro anticipato
- il ciclo di governance completo descritto nella Sezione 9
- swap autonomo delle fee su una pool di liquidità reale, con BNB effettivamente ricevuti dal wallet marketing e dal pool reward dello staking
- il primo burn reale: 44.785.811 DMN rimossi dalla supply totale
- riscossione dei reward su più wallet, quadrata al wei su tre indirizzi
- pausa e riattivazione del guardian
- rifiuto dell'esecuzione anticipata (`TooEarly`) e della doppia esecuzione (`AlreadyExecuted`)
- verifica economica della fee post-esecuzione: esattamente 4,00%

Un ulteriore ciclo è in corso mentre scriviamo: la proposta di governance che trasferirà alla tesoreria i token di migrazione non riscattati è stata proposta e votata, e attende il proprio periodo di timelock. È l'ultima funzione del sistema mai eseguita on-chain.

Il registro completo è pubblicato come `TESTNET_RESULTS.md`.

10.7 Audit esterno

Tutto quanto sopra è stato prodotto dalle stesse persone che hanno scritto il codice. È utile e insufficiente.

I contratti sono congelati al tag `audit-scope-v2` e sottoposti a revisione di sicurezza indipendente. I nostri impegni al riguardo sono tre:

Il report sarà pubblicato integralmente, qualunque cosa contenga. Non una sintesi, non un certificato, non estratti selezionati.

Non faremo il deploy su mainnet prima che sia completato. Nessuna data di lancio è stata annunciata, e nessuna lo sarà finché la revisione non sarà conclusa.

L'auditor riceve la nostra stessa lista di preoccupazioni, incluso il percorso meno provato descritto in 10.4. Un audit serve a trovare quello che ci è sfuggito, non a confermare quello che già sappiamo.

Un bug bounty pubblico è previsto dopo il lancio, così che l'incentivo a segnalare una vulnerabilità superi permanentemente l'incentivo a sfruttarla.

11. Fondi e tesoreria

11.1 Due luoghi, due regole

I fondi del protocollo esistono in due luoghi, governati diversamente, per una ragione che vale la pena spiegare invece di dare per scontata.

La tesoreria detiene la riserva: i token legacy raccolti durante la migrazione, i DMN non riscattati trasferiti dopo la scadenza, e qualsiasi asset la DAO accumuli. È controllata dal timelock. Ogni uscita richiede una proposta, un voto, un ritardo di sette giorni e un'esecuzione. Nessun individuo può muovere un singolo token da essa.

Il wallet operativo riceve in BNB la quota marketing delle fee sulle transazioni. È un wallet multi-firma con firmatari noti, usato per le spese ordinarie: servizi, strumenti, design, infrastruttura.

11.2 Perché non tutto sotto governance

Un protocollo in cui ogni spesa richiede un voto sembra più decentralizzato. Nella pratica è impraticabile: presentare una proposta, aspettare un giorno, condurre un voto di cinque giorni e aspettarne altri sette per pagare un elemento grafico non è governance, è paralisi. I progetti che la adottano o la abbandonano in silenzio o smettono di spendere.

La struttura a due luoghi risolve la tensione senza fingere che non esista:

- Le decisioni grandi — riserve, allocazioni, qualsiasi cosa rilevante — sono votate. Nessuna fiducia richiesta.
- I piccoli costi operativi sono gestiti da un multisig finanziato da una quota di entrate limitata e continuativa. Fiducia richiesta, ma **limitata e rendicontabile**.

Il wallet operativo non detiene mai le riserve del protocollo. Se una chiave di firma venisse compromessa, l'esposizione è il saldo operativo corrente, non la tesoreria. Il caveau resta chiuso in ogni caso.

È la struttura usata dalle DAO mature — una tesoreria governata affiancata da multisig operativi con budget limitati — per le stesse ragioni.

11.3 Cosa la governance controlla qui

Anche il wallet operativo non è fuori dal sistema. L'indirizzo che riceve la quota marketing è impostato da una funzione riservata alla governance: la DAO può reindirizzare quel flusso di entrate in qualsiasi momento, con un voto. Ciò che non fa è approvare ogni singolo pagamento.

Due indirizzi sono permanentemente fuori dalla portata di chiunque, governance inclusa: l'indirizzo morto che riceve i token bruciati, e la tesoreria della migrazione. Entrambi sono `immutable`, fissati al deploy. Alcune destinazioni non dovrebbero essere reindirizzabili da una maggioranza.

11.4 Impegni

I firmatari del wallet operativo saranno pubblici. Le sue spese saranno rendicontabili on-chain, perché ogni transazione che effettua è visibile per costruzione. E se la community concludesse che l'assetto vada cambiato — una quota inferiore, un tetto di spesa imposto nel codice, il controllo pieno della tesoreria — quella è una proposta come le altre.

12. Roadmap

Questa roadmap distingue tra ciò che esiste, ciò che è previsto, e ciò che è una possibilità a lungo termine. Non vengono date date oltre la fase attuale, e nulla di quanto segue costituisce un impegno a realizzare.

Ed è anche, per costruzione, incompleta. La Sezione 12.5 spiega perché.

12.1 Fase 1 — Ora

Il protocollo descritto in questo documento: token, staking, governance, timelock, migrazione. Deployato ed esercitato integralmente su BSC testnet, congelato per l'audit esterno, in attesa della revisione prima del deploy su mainnet.

Completare questa fase significa: audit concluso e pubblicato, deploy su mainnet, finestra di migrazione aperta, liquidità iniziale predisposta.

12.2 Fase 2 — Daimon come protocollo DeFi

Il protocollo attuale è un token con una governance. L'intenzione è che diventi un ecosistema di primitive finanziarie, posseduto e diretto dalle persone che lo usano.

La direzione è deliberata: gli strumenti che determinano cosa accade ai risparmi delle persone comuni — credito, rendimento, liquidità — sono detenuti quasi interamente da istituzioni che ne stabiliscono i termini senza consultazione. Ricostruirli come contratti open source e permissionless governati da chi li usa è la forma pratica dell'argomento esposto nella Sezione 3.

Le primitive che intendiamo perseguire:

Lending e borrowing. Fornire asset per guadagnare interessi, e prendere in prestito contro collaterale. Tassi determinati algoritmicamente dall'utilizzo invece che dalla politica di un'istituzione, requisiti di collaterale visibili nel codice, regole di liquidazione identiche per tutti.

Fornitura di liquidità. Meccanismi per fornire liquidità e guadagnare una quota dell'attività di scambio, con i termini — ripartizione delle fee, struttura degli incentivi, coppie supportate — stabiliti dalla governance invece che da un operatore.

Strategie di rendimento. Prodotti strutturati che indirizzano capitale verso protocolli consolidati, con il profilo di rischio di ciascuna strategia dichiarato esplicitamente invece che sepolto, e la selezione delle strategie decisa dal

voto.

Ulteriori primitive man mano che maturano. Derivati, credito strutturato, meccanismi assicurativi, integrazione di asset del mondo reale, liquidità cross-chain — la superficie della DeFi si espande di continuo, e qualsiasi primitiva costruibile come contratto permissionless è una candidata.

Come questo si collega al token. Ciascuna di queste genera entrate per il protocollo. Quelle entrate entrano nel ciclo che il token già implementa: una quota a buyback e burn, una quota agli staker, una quota alla tesoreria. Il motore economico del protocollo è già costruito; la Fase 2 riguarda il dargli più carburante.

L'ambizione dichiarata è che una persona con pochissimo capitale e nessuna formazione finanziaria possa accedere agli stessi strumenti di chiunque altro, alle stesse condizioni, con le regole visibili e la governance aperta. Questo è l'obiettivo. Se e fino a che punto verrà raggiunto dipende dall'esecuzione, dalle risorse e dal tempo.

Vincoli che si applicheranno. Ogni modulo sarà auditato indipendentemente prima del deploy. Ogni modulo sarà introdotto attraverso la governance invece che annunciato. I moduli che gestiscono fondi degli utenti saranno costruiti con gli stessi vincoli del protocollo principale: nessun proprietario, nessun prelievo privilegiato, nessun parametro senza limite.

12.3 Fase 3 — Una tesoreria attiva

La tesoreria attualmente detiene asset e non ne fa nulla. Una capacità futura permetterebbe alla DAO di allocarli in protocolli approvati, così che il capitale inattivo generi entrate che alimentano lo stesso ciclo.

Meccanicamente si tratta di una chiamata del timelock verso un contratto approvato, in seguito a un voto. La difficoltà non è il meccanismo: una tesoreria in grado di interagire con contratti esterni è la più grande e più attraente superficie d'attacco che un protocollo possa creare. Richiederebbe una allowlist di protocolli approvati invece di chiamate arbitrarie, limiti per operazione, un audit dedicato, e il deploy solo dopo che il protocollo principale abbia operato su mainnet abbastanza a lungo da essere considerato stabile.

Su cosa questo non è. Il capitale che genera rendimento è capitale che corre rischi: fallimento di contratti di terze parti, impermanent loss, condizioni di mercato. I rendimenti possono essere positivi, nulli o negativi. Nulla qui promette rendimento, e qualsiasi strategia sarebbe selezionata dalla community con i suoi rischi dichiarati esplicitamente nella proposta.

12.4 Fase 4 — Infrastruttura

Una possibilità lontana, inclusa perché una roadmap che si ferma all'orizzonte comodo non è una roadmap.

Se l'ecosistema crescesse fino al punto in cui una chain generalista diventasse un vincolo reale — throughput, costi di transazione, funzionalità a livello di protocollo non disponibili su BNB Chain — la community potrebbe valutare il passaggio a un'infrastruttura dedicata.

La forma realistica non è costruire una blockchain da zero. Una chain nuova parte senza validatori e senza sicurezza economica, il che la rende attaccabile indipendentemente da quanto bene sia progettata. La strada percorribile è una chain applicativa o un layer-2 che **eredita** la sicurezza da una rete consolidata invece di tentare di costruire la propria.

Questo è esplicitamente condizionale. Accade solo se una crescita reale lo giustifica, e solo per decisione della community.

Cosa sopravviverebbe. La tokenomics, il modello di governance, il design dello staking, il principio dell'assenza di proprietario — sono logica, e la logica è portabile. Una chain diversa cambierebbe dove il protocollo gira, non cosa fa.

12.5 La roadmap non è fissa

Tutto quanto sopra sarà sbagliato sotto qualche aspetto, ed è previsto che lo sia.

La DeFi non sta ferma. Primitive che oggi non esistono esisteranno fra tre anni; approcci che ora sembrano essenziali verranno superati; rischi non ancora identificati emergeranno. Una roadmap scritta come sequenza fissa di consegne sarebbe obsoleta prima della prima consegna — e, peggio, impegnerebbe il protocollo a un piano invece che a una direzione.

Daimon è costruito per cambiare. Il percorso di aggiornamento esiste, la governance può estendere il sistema, e i vincoli che non possono essere alterati sono deliberatamente pochi e scelti con precisione: nessuna emissione, un floor di supply, un tetto alle fee, un ritardo obbligatorio. Tutto il resto è aperto, perché tutto il resto dovrebbe poter migliorare.

Non è una cautela aggiunta per sicurezza. È il concetto da cui il protocollo prende il nome.

Eraclito — la cui lettura del daimon compare nella Sezione 2 — è anche il filosofo del divenire: *tutto scorre*, nulla resta ciò che era, e ciò che permane lo fa precisamente cambiando. Il fiume persiste perché l'acqua non lo fa. Un sistema progettato per restare identico a sé stesso non sarebbe un'implementazione fedele di quell'idea; ne sarebbe l'opposto.

Quindi la roadmap si muoverà. Nuove frontiere verranno aggiunte man mano che appaiono, e direzioni prese qui verranno abbandonate quando se ne troveranno di migliori. Ciò che non cambierà è il piccolo insieme di vincoli che rendono il protocollo ciò che è — e il requisito che ogni cambiamento passi da un voto pubblico e sette giorni allo scoperto.

La destinazione non è fissa. Il metodo sì.

13. Cosa Daimon non è

La maggior parte dei whitepaper finisce con l'ambizione. Questo finisce con i limiti, perché chi legge per decidere se partecipare è servito meglio sapendo cosa può andare storto che da un altro paragrafo su cosa potrebbe andare bene.

Daimon non è un prodotto d'investimento e non promette rendimenti. Non c'è un obiettivo di rendimento, non c'è una proiezione, non c'è alcuna aspettativa di apprezzamento dichiarata in alcun punto di questo documento. Il protocollo garantisce regole — una supply che non può essere inflazionata, parametri che non possono essere cambiati in privato, un floor che non può essere attraversato. Non garantisce nulla riguardo al prezzo, che è determinato da un mercato che nessun codice controlla.

La deflazione non è un meccanismo di prezzo. Una supply che si riduce non aumenta meccanicamente il valore. La domanda può calare più velocemente dell'offerta. Il floor garantisce scarsità; non garantisce nient'altro.

Non c'è un team che vi proteggerà. È il senso del design, e taglia in entrambe le direzioni. Nessuno può cambiare le regole contro di voi, e nessuno può intervenire per aiutarvi. Non esiste un supporto che possa annullare una transazione, recuperare una chiave persa, o compensare una perdita. Assenza di proprietario significa esattamente quello che dice.

La governance può sbagliare. Una maggioranza può approvare una proposta cattiva. I vincoli nel codice limitano il danno — le fee non possono superare il 10%, la supply non può essere emessa né bruciata sotto il floor, i fondi non possono essere reindirizzati verso indirizzi arbitrari — ma entro quei limiti, la community può decidere male. Il timelock di sette giorni esiste perché un errore sia visibile prima di avere effetto, non perché diventi impossibile.

Il codice può contenere difetti. È stato testato estesamente, analizzato staticamente, attaccato deliberatamente dai suoi stessi autori, e sottoposto a revisione indipendente. Niente di tutto ciò lo rende perfetto. Contratti auditati dalle migliori società del settore sono stati sfruttati. L'affermazione onesta è che abbiamo ridotto la probabilità di fallimento per quanto sappiamo fare, non che l'abbiamo eliminata.

La partecipazione è volontaria e comporta rischi. I token possono perdere valore. I token bloccati non possono essere ritirati in anticipo. Gli smart contract possono fallire. Nulla di quanto scritto qui dovrebbe essere inteso come un consiglio ad acquisire, detenere o mettere in staking alcunché.

Non siamo neutrali. Questo documento è stato scritto dalle persone che hanno costruito il protocollo. Ci crediamo, il che è una ragione per leggere le nostre affermazioni criticamente invece di accettarle. Tutto ciò che viene qui affermato corrisponde a codice pubblico e deployato. La risposta appropriata a questo documento non è fidarsi. È verificare.

14. Avvertenze legali

Questo documento è fornito a scopo puramente informativo. Non costituisce un'offerta di vendita, una sollecitazione all'acquisto, né una raccomandazione riguardante alcun asset digitale, strumento finanziario o titolo.

Nulla in questo documento costituisce consulenza di investimento, finanziaria, legale, fiscale o contabile. Chi legge dovrebbe condurre le proprie ricerche e consultare professionisti qualificati prima di prendere qualsiasi decisione.

Gli asset digitali comportano rischi sostanziali, inclusa la perdita totale dell'importo coinvolto. Le performance passate, i risultati su testnet e le caratteristiche tecniche non indicano né garantiscono esiti futuri. Gli smart

contract possono contenere vulnerabilità nonostante i test e la revisione indipendente.

Daimon è un protocollo decentralizzato senza proprietario, senza amministratore e senza alcuna entità giuridica che lo operi. Nessuna parte garantisce il suo funzionamento continuativo, mantiene un obbligo di supportarlo, o è in grado di annullare, modificare o compensare transazioni eseguite dai suoi contratti.

Le affermazioni contenute in questo documento riguardanti sviluppi futuri rappresentano intenzioni e possibilità, non impegni. Qualsiasi fase futura dipende da decisioni di governance della community, fattibilità tecnica e risorse disponibili.

Il trattamento normativo degli asset digitali varia da giurisdizione a giurisdizione e continua a evolvere. Chi legge è responsabile di determinare se la propria partecipazione sia lecita nel luogo in cui risiede.

Le informazioni contenute in questo documento sono accurate al meglio delle conoscenze degli autori alla data di pubblicazione. Indirizzi dei contratti, parametri e dettagli tecnici dovrebbero essere verificati direttamente on-chain, che resta in ogni caso la fonte autorevole.

Repository: github.com/daimon-dao/daimon-dao **Scope dell'audit:** tag [audit-scope-v2](#)

Bozza v0.1 — in attesa dell'audit esterno. Questo documento sarà aggiornato per riflettere l'esito dell'audit prima della pubblicazione.